



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH



CE433 - SoC Design

GIỚI THIỆU MÔN HỌC

CBGD: ThS Trương Văn Cường

Email: cuongtv@uit.edu.vn

Thông tin môn học (Đề Cương)

- Trình bày khái niệm về hệ thống SoC.
- Cung cấp các kiến thức nền tảng về các thành phần cơ bản của một hệ thống SoC như
processor, memory, interconnect.
- Trình bày những vấn đề phải đối mặt trong quá trình thiết kế hệ thống như về diện tích, tốc độ, công suất tiêu thụ và khả năng cấu hình được.
- Phân tích một số hệ thống SoC cho các ứng dụng khác nhau.

MỤC TIÊU MÔN HỌC

Ký hiệu	Mục tiêu môn học ^[1]	Chuẩn đầu ra trong CTĐT ^[2]
<i>G1</i>	Phân tích và tổng hợp bài toán ngành kỹ thuật máy tính và áp dụng vào học tập suốt đời trong lĩnh vực Kỹ thuật máy tính.	<i>5</i>
<i>G2</i>	Thiết kế, xây dựng được các ứng dụng trong lĩnh vực Kỹ thuật máy tính.	<i>10.2</i>
<i>G3</i>	Nhận diện được các thuật ngữ chuyên môn bằng ngoại ngữ.	<i>9.2</i>
<i>G4</i>	Có kỹ năng viết báo cáo kỹ thuật chuyên ngành.	<i>8.1</i>

CHUẨN ĐẦU RA MÔN HỌC

CĐRMH [1]	Mô tả CĐRMH (Mục tiêu cụ thể) [2]	Mức độ giảng dạy[3]
<i>G1.1 (5.1.1)</i>	Áp dụng được các phương pháp tính toán để xác định các thông số yêu cầu của một hệ thống SoC.	<i>ITU</i>
<i>G1.2 (5.2)</i>	Áp dụng vào học tập suốt đời trong lĩnh vực Kỹ thuật máy tính.	<i>I</i>
<i>G2 (10.2.2)</i>	Thiết kế được các thành phần cơ bản của một hệ thống SoC tiêu biểu.	<i>ITU</i>
<i>G3(9.2.1)</i>	Nhận diện được các thuật ngữ chuyên môn về hệ thống SoC.	<i>IT</i>
<i>G4(8.1.1)</i>	Mô tả được hệ thống SoC cơ bản.	<i>I</i>

Mục tiêu môn học

- Hiểu SoC là gì và vì sao cần SoC
- Phân biệt CPU – MCU – SoC
- Hiểu vai trò của FPGA trong thiết kế SoC
- Biết nội dung & lộ trình môn học
- Hiểu kỹ năng đạt được sau môn học

KẾ HOẠCH GIẢNG DẠY (12 tuần)

- Chương 1: Giới thiệu về thiết kế hệ thống SoC trên FPGA
- Chương 2: Kiến trúc vi xử lý trên hệ thống SoC của FPGA
- Chương 3: Bộ nhớ
- Chương 4: Hệ thống kết nối
- Chương 5: Các tiêu chuẩn đánh giá hệ thống SoC
- Chương 6: Tổng hợp cấp cao trong thiết kế SoC trên FPGA

Quy định học tập

- Dự lớp đầy đủ, xem trước tài liệu
- Tham gia thảo luận, trả lời câu hỏi, làm bài tập trên lớp
- Làm bài tập về nhà đúng hạn
- Chủ động liên hệ giảng viên khi có thắc mắc

Tài liệu + Tool

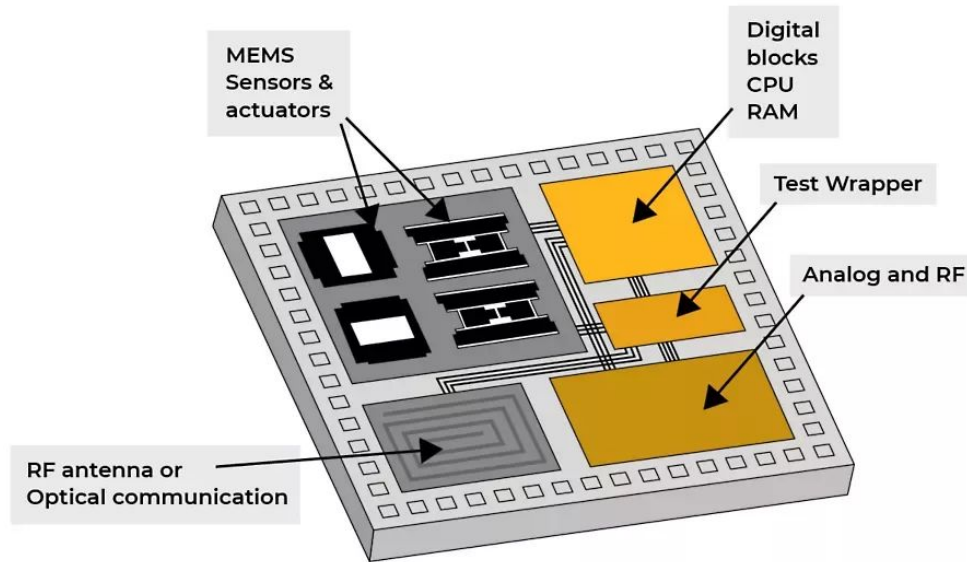
Giáo trình

1. THIẾT KẾ HỆ THỐNG SoC TRÊN FPGA (TS. Trần Thị Điểm)

Tools:

- Altera (Web Edition version). ModelSim-Altera Starter Edition (>10.1b) for Quartus II (>v12.1).
- Litex
- Chipyard

What is a System on a Chip (SoC)?



Hệ thống trên chip (SoC) là một mạch tích hợp (IC) kết hợp tất cả các thành phần thiết yếu của một hệ thống lên trên một miếng silicon duy nhất, loại bỏ nhu cầu sử dụng các bộ phận hệ thống rời rạc, công kênh. Việc tích hợp này giúp đơn giản hóa thiết kế mạch in và mang lại hiệu quả năng lượng cũng như tốc độ xử lý cao hơn, đồng thời vẫn đảm bảo đầy đủ chức năng.

SoC khác gì CPU và MCU?

Đặc điểm	CPU	MCU	SoC
Bộ xử lý	✓	✓	✓
Bộ nhớ tích hợp	✗	✓	✓ (đa cấp)
Bus nội	✗	Hạn chế	✓
IP ngoại vi	✗	✓	✓✓
Hệ điều hành	✓	✗ / RTOS	✓
Mức độ tích hợp	Thấp	Trung bình	Cao

Thành phần & Cấu trúc của SoC

- Data processing units
- Embedded memory
- Graphics processing units (GPUs)
- AI Accelerators: TPU, NPU
- Video and audio processing
- Peripherals

Thành phần & Cấu trúc của SoC

- CPU, MCU, DSP hoặc bộ xử lý chuyên dụng (ASIP)
- RAM, ROM, Flash, EEPROM, Cache
- HDMI, USB, SPI, I²C, UART, Ethernet...
- Wi-Fi, Bluetooth, RF
- GPU (và các accelerator chuyên dụng)
- Bộ ổn áp, PLL, oscillator, timer, ADC
- Kết nối nội chip:
- Bus nội bộ hoặc Network-on-Chip (NoC)
- Digital, Analog, Mixed-signal cho sensor & actuator

*Tích hợp cao – hiệu
năng tốt – tiêu thụ
năng lượng thấp*

Vai trò & Xu hướng phát triển của SoC

- Giảm tiêu thụ năng lượng, chi phí và kích thước
- Ít sinh nhiệt → thiết kế tản nhiệt hiệu quả
- Độ trễ thấp, truyền dữ liệu nhanh
- Ứng dụng tiêu biểu
- Smartphone, IoT, hệ thống nhúng
- Ô tô thông minh, thiết bị công nghiệp
- PC & Laptop tiết kiệm năng lượng

Vai trò & Xu hướng phát triển của SoC

- Xu hướng hiện đại
- SoC chuyên biệt hóa cho từng bài toán
- Tăng cường AI / Machine Learning / Edge Computing
- Tính toán song song phân tán → vượt giới hạn CPU truyền thống

Lịch sử phát triển của System-on-Chip (SoC)

- **Thập niên 1970:** Khái niệm SoC lần đầu trở thành hiện thực. Năm 1974, SoC đầu tiên xuất hiện trong đồng hồ LCD, tích hợp nhiều chức năng trên một chip thay vì các vi mạch rời rạc.
- **Thập niên 1980–1990:** Công nghệ bán dẫn phát triển mạnh cho phép tích hợp nhiều khối chức năng hơn. SoC bắt đầu hỗ trợ mixed-signal, xử lý cả tín hiệu analog và digital trên cùng một chip.
- **Giai đoạn 2000–2010:** SoC tích hợp Wi-Fi, Bluetooth, modem di động, cùng CPU và GPU mạnh mẽ, đặt nền móng cho smartphone và thiết bị di động hiện đại.
- **Hiện nay:** SoC ngày càng chuyên biệt hóa, mở rộng sang ô tô, thiết bị đeo, công nghiệp, tự động hóa. Các tính năng nổi bật gồm AI, Machine Learning và Edge Computing.

Ứng dụng SoC trong công nghiệp, ô tô & y tế

- Hệ thống ô tô (Automotive Systems):
 - Dẫn đường, infotainment, cảm biến, hệ thống tránh va chạm
- Ứng dụng công nghiệp (Industrial Applications):
 - Xử lý thời gian thực, kết nối công nghiệp, điều khiển & tự động hóa
- Thiết bị y tế (Medical Devices):
 - Theo dõi bệnh nhân, thiết bị chẩn đoán, thiết bị cấy ghép thông minh
- Ứng dụng đa dạng khác:
 - Từ đồ chơi trẻ em đến hệ thống máy móc công nghiệp

Ứng dụng SoC trong thiết bị tiêu dùng & kết nối

- Thiết bị di động (Mobile Devices):
 - Smartphone, tablet – tích hợp CPU, GPU, multimedia và kết nối không dây
- Internet of Things (IoT):
 - Wearables, smart home – tối ưu năng lượng thấp, kích thước nhỏ
- Thiết bị mạng (Networking Equipment):
 - Router, switch – xử lý gói tin, bảo mật, định tuyến dữ liệu hiệu quả
- Điện tử tiêu dùng (Consumer Electronics):
 - Máy chơi game, media player – xử lý đồ họa và kết nối đa phương tiện

Ưu điểm của System-on-Chip (SoC)

- Tối ưu không gian (Space Optimization):
 - Tích hợp nhiều khối chức năng trên một chip → thiết bị nhỏ gọn hơn, thiết kế linh hoạt hơn.
- Hiệu quả năng lượng (Power Efficiency):
 - Giảm số lượng linh kiện rời → tiêu thụ điện thấp, đáp ứng tốt các chỉ tiêu PPA (Power–Performance–Area).
- Chi phí thấp (Lower Cost):
 - Một chip SoC rẻ hơn so với việc sử dụng nhiều IC riêng lẻ.
- Độ tin cậy cao (Reliability):
 - Ít kết nối vật lý → giảm lỗi phần cứng, tăng độ ổn định hệ thống.
- Hiệu năng cao (Performance):
 - Truyền tín hiệu nội chip → tốc độ nhanh hơn, độ trễ thấp hơn so với hệ thống đa chip.

Nhược điểm của System-on-Chip (SoC)

- Điểm lỗi duy nhất (Single Point of Failure):
 - Hỏng một khối chức năng có thể làm toàn bộ hệ thống ngừng hoạt động, khó nâng cấp hay thay thế từng phần.
- Thời gian ra thị trường dài (Time to Market):
 - Thiết kế SoC tùy biến đòi hỏi chuyên môn cao, công cụ đặc thù → chi phí & thời gian phát triển lớn, chỉ hiệu quả khi sản xuất số lượng lớn.
- Hạn chế analog (Mixed Analog/Digital):
 - Dùng một công nghệ chế tạo cho toàn chip → phần analog không đạt hiệu suất tối ưu, SoC phù hợp hơn với ứng dụng số (digital).
- Tính linh hoạt thấp (Limited Flexibility):
 - SoC được tối ưu cho một nhiệm vụ cụ thể, khó tái sử dụng cho các ứng dụng khác.

HOẠT ĐỘNG THẢO LUẬN NHÓM

- Hình dung SoC là gì trước khi học design flow
- Kích hoạt tư duy hệ thống (system-level thinking)
- SoC Design Flow trên FPGA

HOẠT ĐỘNG THẢO LUẬN NHÓM

- Theo bạn, một hệ thống SoC tối thiểu cần những thành phần nào?
- Vì sao CPU đơn lẻ là chưa đủ cho các ứng dụng nhúng/AI?
- Nếu triển khai trên FPGA:
 - Phần nào nên làm bằng phần cứng?
 - Phần nào nên để phần mềm xử lý?
 - Theo bạn, khó khăn lớn nhất khi thiết kế SoC là gì?

HOẠT ĐỘNG THẢO LUẬN NHÓM

 Thời gian: **10–15 phút**

 Thảo luận nhóm

 Kết quả:

- Liệt kê các khối chức năng **hoặc**
- Vẽ sơ đồ SoC đơn giản theo hiểu biết ban đầu

Quy trình thiết kế System-on-Chip (SoC)

1. Specification
2. Logical Design (HDL + Simulation)
3. Logic Synthesis (Netlist)
4. Physical Design (Place & Route)
5. Signoff (Timing / Power / DRC / LVS)
6. Tapeout
7. Manufacturing
8. Testing & Packaging

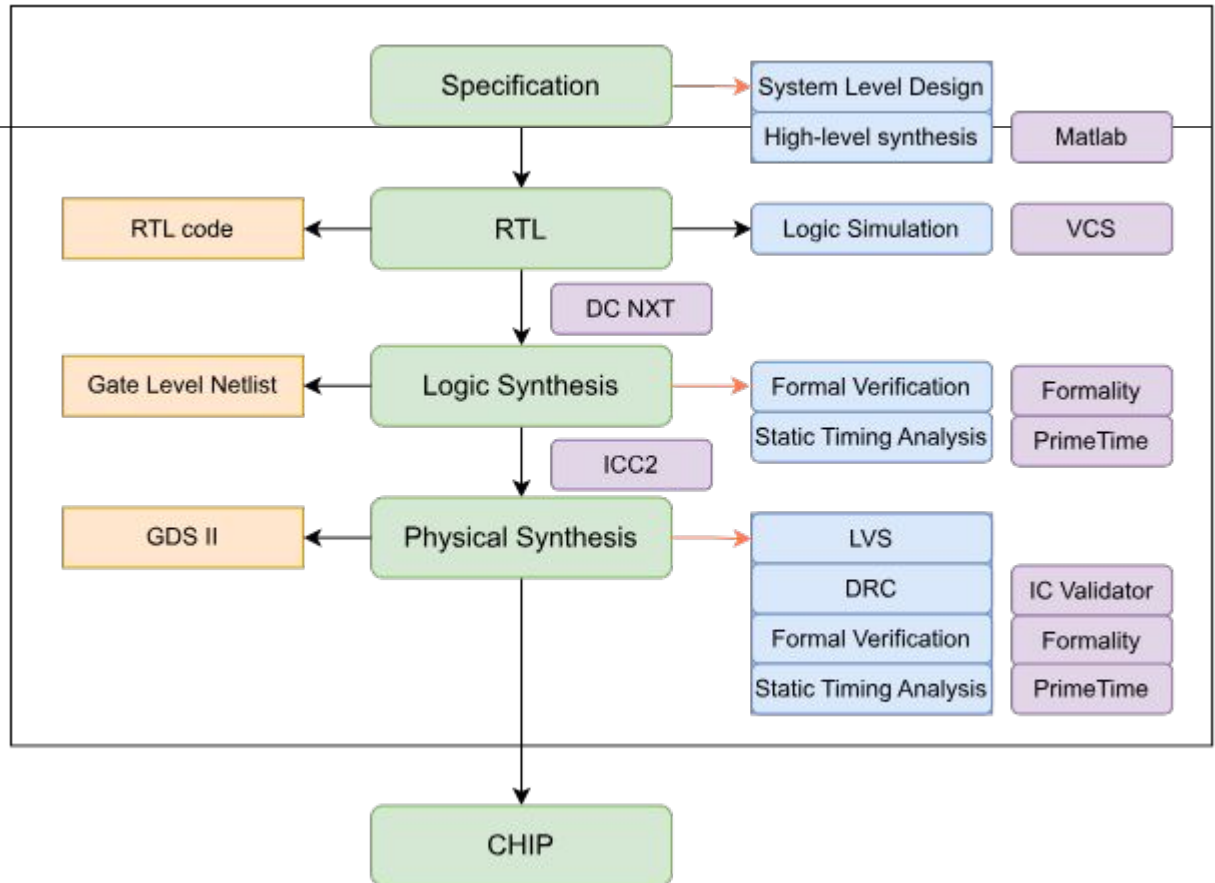
Quy trình thiết kế System-on-Chip (SoC)

- Specification (Đặc tả):
 - Xác định rõ chức năng, ứng dụng, mục tiêu hiệu năng, giới hạn năng lượng, chi phí.
- Logical Design (Thiết kế logic):
 - Mô tả hành vi hệ thống bằng HDL (Verilog/VHDL/SystemC) và mô phỏng để kiểm tra tính đúng đắn.
- Logic Synthesis (Tổng hợp logic):
 - Chuyển mô tả HDL thành netlist gồm các phần tử logic và kết nối giữa chúng.
- Mục tiêu: đảm bảo thiết kế đúng chức năng trước khi đi sâu vào phần vật lý

Từ thiết kế vật lý đến sản xuất SoC

- Physical Design (Thiết kế vật lý):
 - Lựa chọn transistor, bố trí (placement) và định tuyến dây (routing) trên silicon.
- Signoff (Kiểm chứng cuối):
 - Phân tích timing, công suất, độ tin cậy, khả năng chế tạo (DFM).
 -  Lỗi ở giai đoạn này → toàn bộ chip sản xuất phải bỏ.
- Tapeout:
 - Xuất dữ liệu layout cuối cùng (photomask) gửi nhà máy sản xuất.
- Testing & Packaging:
 - Kiểm thử chip theo đặc tả và đóng gói để sẵn sàng đưa vào sản phẩm.

Synopsys (ref)



Quy Trình Thiết Kế SoC trên FPGA

1. Specification
2. Architecture
3. IP Design
4. Integration
5. Verification
6. Implementation
7. Software
8. Optimization

Quy Trình Thiết Kế SoC trên FPGA

1. Specification

- Ứng dụng gì? (control, DSP, AI, vision...)
- Có cần CPU không? → soft CPU hay hard CPU?
- Yêu cầu:
 - Throughput
 - Latency
 - Power
 - Tài nguyên FPGA

Architecture Design

- Architecture Design (Thiết kế kiến trúc)
- CPU
 - Soft CPU: MicroBlaze / Nios II / RISC-V
 - Hard CPU (nếu có): ARM Cortex-A (Zynq, Agilex SoC...)
- Memory
 - BRAM (on-chip)
 - DDR (off-chip, nếu dùng SoC FPGA)
- Interconnect
 - AXI (Xilinx)
 - Avalon (Intel)
 - Wishbone (LiteX, RISC-V học thuật)
- IP Blocks
 - GPIO
 - Timer
 - DMA
- Custom accelerator (RTL / HLS)

IP Design

- RTL Design
 - Verilog / VHDL
 - Tự viết IP (GPIO, Timer, Accelerator)
 - Kiểm soát timing & tài nguyên tốt nhất
- HLS (High-Level Synthesis)
 - C / C++ / SystemC → RTL
 - Dùng cho:
 - DSP
 - AI kernel
 - Image processing
- C. Vendor IP
 - AXI GPIO
 - AXI TimerDDR ControllerEthernet MAC

SoC Integration (Tích hợp hệ thống)

- Kết nối IP vào bus
 - Gán địa chỉ (memory map)
 - Clock & Reset domain
 - Interrupt
- Tool:
 - Vivado Block Design
 - Platform Designer (Quartus)
 - LiteX SoC Builder

Functional Verification

- RTL Simulation (ModelSim / XSim)
- SoC simulation:
 - CPU chạy firmware
 - Read/write IP registers
- Với HLS:
 - C simulation
 - C/RTL co-simulation

Synthesis & Implementation

- Synthesis → LUT / FF / BRAM / DSP
- Place & Route
- Timing Analysis:
 - Xác định tần số tối đa có thể chạy được
 - Phát hiện đường tín hiệu quá chậm
 - Đảm bảo thiết kế không vi phạm timing trước khi triển khai trên FPGA / ASIC
- FPGA là nơi đánh đổi
 - Area ↔ Frequency ↔ Power

Bitstream Generation & FPGA Programming

- Generate bitstream
- Program FPGA qua JTAG / SD / Flash
- SoC chạy thật trên board

Software/Firmware Development

- Song song với phần cứng:
 - Bare-metal (C)
- RTOS (FreeRTOS)
- Linux (nếu hard CPU)
- Quan trọng:
 - Driver cho IP tự thiết kế
 - Memory-mapped I/O
 - Interrupt handling

Bare-metal (C)

- Đặc điểm
 - Code C chạy trực tiếp trên CPU, không có OS, không scheduler.
 - Lập trình bằng cách:
 - Khởi tạo clock, RAM, stack
 - Cấu hình peripheral qua memory-mapped registers
 - Viết while(1) + interrupt handler
 - Toàn quyền kiểm soát phần cứng.
- Ưu điểm
 -  Độ trễ thấp nhất, timing xác định (deterministic)
 -  Tốn ít tài nguyên nhất (RAM/Flash rất nhỏ)
 -  Phù hợp MCU nhỏ / soft-CPU trên FPGA
 -  Dễ debug ở mức thanh ghi

Bare-metal (C)

- Nhược điểm
 -  Không có multitasking thật sự
 -  Khó mở rộng khi hệ thống lớn
 -  Quản lý state phức tạp (FSM lớn)
- Phù hợp cho
 - Bootloader
 - Điều khiển GPIO, Timer, UART
 - IP test trên FPGA
 - Hard real-time cực ngắn
-  Ví dụ
 - RISC-V softcore + GPIO
 - MicroBlaze / Nios II điều khiển LED, PWM

RTOS (FreeRTOS)

- Đặc điểm
 - Có scheduler, task, priority
 - Hỗ trợ:
 - Semaphore / Mutex
 - Queue / EventGroup
 - Software timer
 - Vận hành rất gần phần cứng (no MMU)
- Ưu điểm
 -  Real-time xác định, có ưu tiên
 -  Tách code theo task logic
 -  Dễ bảo trì hơn bare-metal
 -  Overhead thấp (vài KB RAM)

RTOS (FreeRTOS)

- Nhược điểm
 -  Không có process isolation
 -  Không có virtual memory
 -  Debug phức tạp hơn bare-metal
- Phù hợp cho
 - Điều khiển nhiều tác vụ song song
 - Giao tiếp (UART, SPI, CAN, Ethernet)
 - Soft-CPU hoặc hard-CPU nhỏ
 - Edge/IoT real-time
-  Ví dụ
 - FreeRTOS trên Cortex-M
 - FreeRTOS trên RISC-V softcore
 - FreeRTOS điều phối CPU ↔ IP accelerator

Linux (nếu hard CPU)

- Điều kiện
 - CPU đủ mạnh:
 - ARM Cortex-A (A9, A53, A72...)
 - Có MMU
 - RAM $\geq$ vài chục MB
 - Storage (SD/eMMC/NAND)
- Đặc điểm
 - User space / Kernel space
 - Driver model chuẩn
 - Virtual memory, filesystem, networking
 - Hỗ trợ container, AI runtime, v.v.
- Ưu điểm
 -  Phát triển nhanh
 -  Hệ sinh thái khổng lồ
 -  Chạy AI, networking, UI
 -  Phù hợp hệ thống phức tạp

Linux (nếu hard CPU)

- Nhược điểm
 -  Không hard real-time (trừ PREEMPT_RT)
 -  Overhead lớn
 -  Boot chậm hơn RTOS
- Phù hợp cho
 - SoC FPGA có CPU lõi cứng
 - Edge-AI, Vision, Gateway
 - Hệ thống cần update OTA
 - Ứng dụng mạng, web, AI
-  Ví dụ
 - Zynq / Zynq UltraScale+
 - SoC ARM + FPGA
 - Edge AI box, NPU/DPU control

Tiêu chí	Bare-metal	RTOS	Linux
OS	✗	RTOS	Full OS
Multitask	✗	✓	✓
Real-time	★★★★★	★★★★	★
Resource	★★★★★	★★★★	★
Complexity	★	★★	★★★★
Use case	Điều khiển thấp	Embedded RT	Edge / AI / Gateway

System Evaluation & Optimization

- Đánh giá:
 - Tài nguyên
 - Hiệu năng
 - Năng lượng
 - Độ tin cậy
- Tối ưu:
 - Pipeline
 - DMA
 - Accelerator offload
 - HLS pragma

SoC With Litex

- CPU: VexRiscv (khuyến nghị), hoặc Rocket/LM32 (ít dùng)
- Bus: LiteX dùng Wishbone nội bộ (có bridge sang AXI/Avalon nếu cần)
- Memory: Integrated SRAM/ROM + (tùy board) DDR/SDRAM
- Peripherals: UART, Timer, GPIO, SPI, I2C, Ethernet...
- Tool mục tiêu: mô phỏng (Verilator) hay chạy thật (FPGA board)

Architecture → SoC class (Python) trong LiteX

- Trong LiteX, “kiến trúc” = Python code tạo SoC:
 - SoCCore(...) (xương sống: CPU, bus, CSR, memory map)
 - add_* peripheral (UART/Timer/GPIO...)
 - CSR map / memory map tự sinh (LiteX sinh file header cho firmware)
 - Output: 1 file Python kiểu soc.py (hoặc base.py) mô tả SoC.

IP design

- A) RTL Custom Peripheral
 - Viết Verilog/VHDL (hoặc Migen/nmigen)
 - Gắn vào bus bằng LiteX helper:
 - AutoCSR + CSRStorage/CSRStatus
 - hoặc làm “Wishbone slave” trực tiếp
- B) Migen/nmigen logic
 - Viết phần cứng bằng Python DSL (Migen) → sinh Verilog
- C) HLS (tùy chọn)
 - Bạn có thể dùng Vitis HLS / Intel HLS rồi import RTL netlist vào LiteX,
 - nhưng “LiteX-native” thường khuyến nghị RTL/Migen cho môn học.
 - Output: module IP + register interface (CSR/WB).

Integration: “add peripheral + CSR + address map”

- Tương đương “Block Design” nhưng bằng code:
 - `self.add_csr("myip")`
 - `self.bus.add_slave(...)` hoặc `self.add_peripheral(...)`
- LiteX tự sinh:
 - `csr.h`, `mem.h`
 - map địa chỉ CSR, IRQ
 - Output: SoC hoàn chỉnh + header map cho firmware.

Verification → Verilator + litex_sim (mô phỏng SoC chạy firmware)

- Kiểm chứng SoC-level.
 - Mô phỏng RTL bằng Verilator
 - Boot ROM + chạy firmware RISC-V (C) trong sim
 - Quan sát UART log, CSR read/write, waveform (nếu cần)
 - Output: Pass/fail theo testcase (firmware tự test peripheral).

Synthesis/Implementation → build gateway bằng LiteX

- LiteX gọi tool theo FPGA vendor:
 - Xilinx: Vivado
 - Intel: Quartus
 - Lattice: Radiant/diamond...
 - ECP5: yosys/nextpnr (nếu setup)

Software → “Bare-metal RISC-V firmware”

- LiteX đi kèm flow firmware:
 - Toolchain RISC-V GCC
 - BSP headers tự sinh: CSR addresses, IRQ, mem layout
- Viết driver theo CSR:
 - Ví dụ kiểu:
 - `csr_write(...)`
 - `csr_read(...)`
 - Output: firmware.bin/elf + chạy trên SoC thật hoặc simulation.

Bring-up & Debug → UART first, then JTAG

- UART boot log (đơn giản nhất)
- test GPIO/Timer
- đo latency/tốc độ (cycle counter)
- nếu có: OpenOCD/JTAG debug

Thảo luận nhóm (Group Discussion)

- Củng cố kiến thức về SoC design flow trên FPGA
- Rèn luyện tư duy thiết kế hệ thống (system-level thinking)
- Hiểu rõ đánh đổi (trade-off) giữa hiệu năng – tài nguyên – độ phức tạp

Thảo luận nhóm (Group Discussion)

- Với một bài toán cụ thể (ví dụ: GPIO + Timer / Edge-AI đơn giản) (nhóm tự chọn)
- Kiến trúc SoC nên gồm những thành phần nào?
- Nên chọn:
 - CPU mềm (RISC-V) hay không dùng CPU?
 - IP viết RTL hay dùng HLS?
 - Bus & giao tiếp nội bộ:
 - Vì sao chọn bus đó?
- Theo bạn, nút thắt hiệu năng (bottleneck) nằm ở đâu?

Thảo luận nhóm (Group Discussion)

-  Thời gian: **15–20 phút**
-  Làm việc theo nhóm
-  Kết quả:
 - Vẽ sơ đồ SoC (block diagram) **hoặc**
 - Trình bày ngắn gọn bằng gạch đầu dòng

Thảo luận

Vì sao chip điện thoại **được gọi là SoC?**

Thảo luận

- Một chip điện thoại (ví dụ Snapdragon, Apple A-series...) tích hợp gần như toàn bộ hệ thống vào 1 die:
 - CPU (big + little cores)
 - GPU
 - NPU / AI accelerator
 - ISP (camera)
 - DSP (audio, signal)
 - Memory controller
 - I/O, display, multimedia
 - Modem (nhiều đời tích hợp luôn)

Thảo luận

- 👉 Cả hệ thống = 1 chip
- 👉 Không cần chipset rời
- 👉 Đúng 100% định nghĩa System-on-Chip

Thảo luận

CPU Intel / AMD? Có phải SoC không?

Thảo luận

- Trong lịch sử PC:
 - CPU: chỉ lo tính toán
 - Northbridge / Southbridge:
 - Memory controller
 - PCIe
 - I/O, USB, SATA...
- 🙏 Hệ thống = nhiều chip
- 🙏 CPU không phải SoC

Thảo luận

SoC là khái niệm kiến trúc. **CPU** là vai trò chức năng.

Một chip có thể **vừa là CPU, vừa là SoC**, tùy mức độ tích hợp.

Mọi SoC đều có CPU, nhưng không phải mọi CPU đều là SoC